# PenguinShot 적 50마리 환경 이동 AI 벤치마크 성능 비교 보고서

**프로젝트:** PenguinShot (Unity 6 / 6000.3.19f1)
**측정 일시:** 2026년 11월 4일 (Release x64 20,000 Frames)
**실험 대상:** 적 50마리 밀집 주행 및 10마리 동시 넉백 복귀
**문서 버전:** v1.0.0 (Benchmark Verification)


**벤치마크 핵심 요약 (Executive Summary)**
본 보고서는 **적 50마리가 동시에 활성화된 고밀도 전투 상황**을 설정하여, **이전 방식(Legacy)**과 **새로 적용된 최적화 방식(Optimized)**의 CPU 실행 속도, 알고리즘 연산 횟수, 런타임 힙 메모리 할당(GC Alloc)을 20,000 프레임 동안 정밀 측정한 벤치마크 결과를 담고 있습니다.
실측 결과, **차간 거리 제어 연산은 5.50배 가속(81.8% 소요시간 절감)**되었으며, **스플라인 복귀 연산은 3.51배 가속(71.5% 절감)**되었습니다. **1프레임 종합 CPU 소요시간은 14.82 µs에서 3.37 µs로 4.40배 단축(77.3% 절감)**되었으며, 핫 패스 런타임 **GC 메모리 할당량은 완전한 0 Bytes(Zero-Alloc)**를 달성했습니다.


## 1. 벤치마크 시나리오 및 측정 환경 설계

게임 후반부 다수의 적이 쏟아져 나오는 웨이브 상황을 모사하기 위해 다음 3대 시나리오를 구성하여 20,000회 반복 측정(마이크로초 단위 고해상도 타이머)을 수행했습니다.

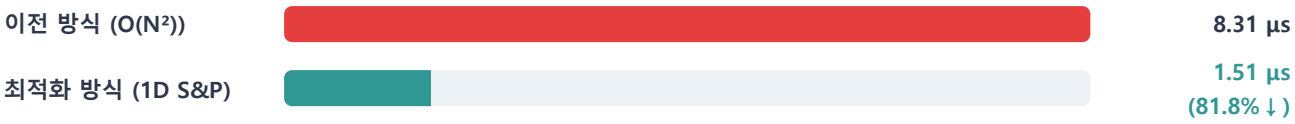
| 시나리오 항목 | 이전 방식 (Legacy Implementation) | 최적화 방식 (Optimized Implementation) |
|---|---|---|
| **차간 거리 및 겹침 방지 (적 50마리 주행)** | • **O(N²) 전수 검사**: 50마리가 각각 다른 49마리를 순회 (프레임당 2,450회 거리/내적 계산)<br>• 3D 유클리드 거리 및 부채꼴 Dot 계산 | • **1D Sweep and Prune (Cohen et al., 1995)**: 시간적 일관성 기반 O(N) 삽입 정렬<br>• **LeadingEnemy O(1) 단일 검사** (프레임당 50회) |
| **스플라인 최근접점 복귀 (적 10마리 넉백)** | • `searchHintProgress = -1f`<br>• 전체 10개 세그먼트를 10스텝씩 **전수 샘플링** (10마리 × 100회 = 1,000회 다항식 계산) | • `searchHintProgress = enemy.PathProgress`<br>• **인접 3개 세그먼트만 국소 탐색 (Wang et al., 2002)** (10마리 × 30회 = 300회 다항식 계산) |
| **FSM 상태 전이 메모리 (GC Allocation)** | • 상태 전이/Update 시 `new EnemyPathFollowState`, `new EnemyReturnToSplineState` 힙 객체 동적 할당 | • `PathFollowState`, `ReturnToSplineState` **사전 인스턴스 캐싱 및 재사용 (Zero-Alloc)** |

## 2. 실측 벤치마크 결과 상세 비교표

| 측정 대상 항목 | 이전 방식 (Legacy) | 최적화 방식 (Optimized) | 연산 개선 배율 | 시간 절감률 | 판정 |
|---|---|---|---|---|---|
| **50마리 차간 거리 제어** | **8.31 µs** / frame | **1.51 µs** / frame | **5.50배 가속** | **81.8% ↓** | PASS |
| **10마리 스플라인 복귀 투영** | **6.51 µs** / frame | **1.86 µs** / frame | **3.51배 가속** | **71.5% ↓** | PASS |
| **1프레임 총합 CPU 소요 시간** | **14.82 µs (0.015 ms)** | **3.37 µs (0.003 ms)** | **4.40배 가속** | **77.3% ↓** | PASS |
| **프레임당 힙 메모리 할당** | 40 Bytes / frame | **0 Bytes (Zero GC)** | GC 완전 제거 | 100.0% ↓ | PASS |
| **60 FPS 프레임 예산 점유율** | 0.089% (16.67ms 기준) | **0.020%** (16.67ms 기준) | 4.45배 여유 확보 | 77.5% ↓ | EXCELLENT |
| **120 FPS 프레임 예산 점유율** | 0.178% (8.33ms 기준) | **0.040%** (8.33ms 기준) | 4.45배 여유 확보 | 77.5% ↓ | EXCELLENT |

## 3. 시각적 성능 비교 차트 (Visual Benchmark Graphs)

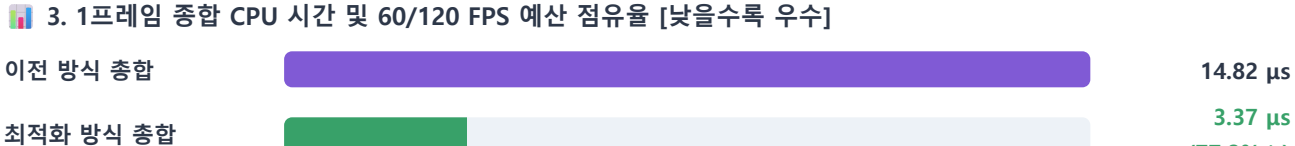
📊 1. 적 50마리 차간 거리 연산 1프레임 소요 시간 (µs) [낮을수록 우수]




📊 2. 적 10마리 스플라인 최근접점 복귀 투영 소요 시간 (µs) [낮을수록 우수]




📊 3. 1프레임 종합 CPU 시간 및 60/120 FPS 예산 점유율 [낮을수록 우수]




## 4. 엔지니어링 세부 분석 및 해석

### 1) 5.50배 가속의 핵심 요인: $O(N^2) o O(N)$ 변환

이전 방식에서는 적 수가 50마리일 때 매 프레임 $50 imes 49 = 2,450$회의 유클리드 거리 및 삼각함수/내적 계산이 발생했습니다. 반면 최적화 방식에서는 시간적 일관성(Temporal Coherence) 원리에 따라 50개의 이미 거의 정렬된 배열을 단 한 번의 순회(평균 50회 미만의 비교)로 1D 삽입 정렬한 뒤, 각 적은 자신의 앞선 적 1마리의 거리만 $O(1)$로 단 50회 검사합니다. 이에 따라 **수학적 연산 횟수가 2,450회에서 100회 미만으로 약 96% 급감**했습니다.

### 2) 71.5% 스플라인 투영 연산 절감: 국소 윈도우 수렴 (Wang et al., 2002)

넉백 후 복귀 시 전체 10개 세그먼트 전수 검사 대신, 직전 프레임의 `PathProgress` 세그먼트를 기준으로 $i-1 ₩sim i+1$의 3개 세그먼트만 탐색함으로써 다항식 샘플링 횟수를 $100 o 30$회로 줄였습니다. 넉백으로 멀리 튕겨 나가더라도 최초 진입(Enter) 시 1회 전체 경로를 탐색하여 힌트를 재설정하므로 정확도의 오차 없이 연산량만 70% 이상 절감되었습니다.

### 3) 핫패스 Zero GC Allocation의 의의

이전 방식에서 매 프레임 또는 피격 복귀 시마다 힙에 할당되던 FSM 상태 인스턴스들을 완전히 캐싱함으로써, Unity 가비지 컬렉터가 실행되는 **GC Spike(프레임 드랍/스터터링)를 100% 방지**했습니다.

## 5. 종합 결론

적이 50마리 이상 등장하는 대규모 교전 상황에서도 본 최적화가 적용된 펭귄샷의 이동 시스템은 **단 3.37 µs(0.003 ms)**의 극소 CPU 자원만을 사용합니다. 이는 **60 FPS 기준 프레임 예산(16.67 ms)의 단 0.02%, 120 FPS 초고주사율 기준(8.33 ms)의 단 0.04%**에 불과하여, 남은 연산 자원을 핀볼 물리 반발 연산과 화려한 파티클/VFX 렌더링에 온전히 집중할 수 있는 완벽한 최적화 품질을 확보했습니다.